

System Design: TinyURL

URL Shortening Service — Senior Engineer Interview Walkthrough

1. Requirements & Capacity

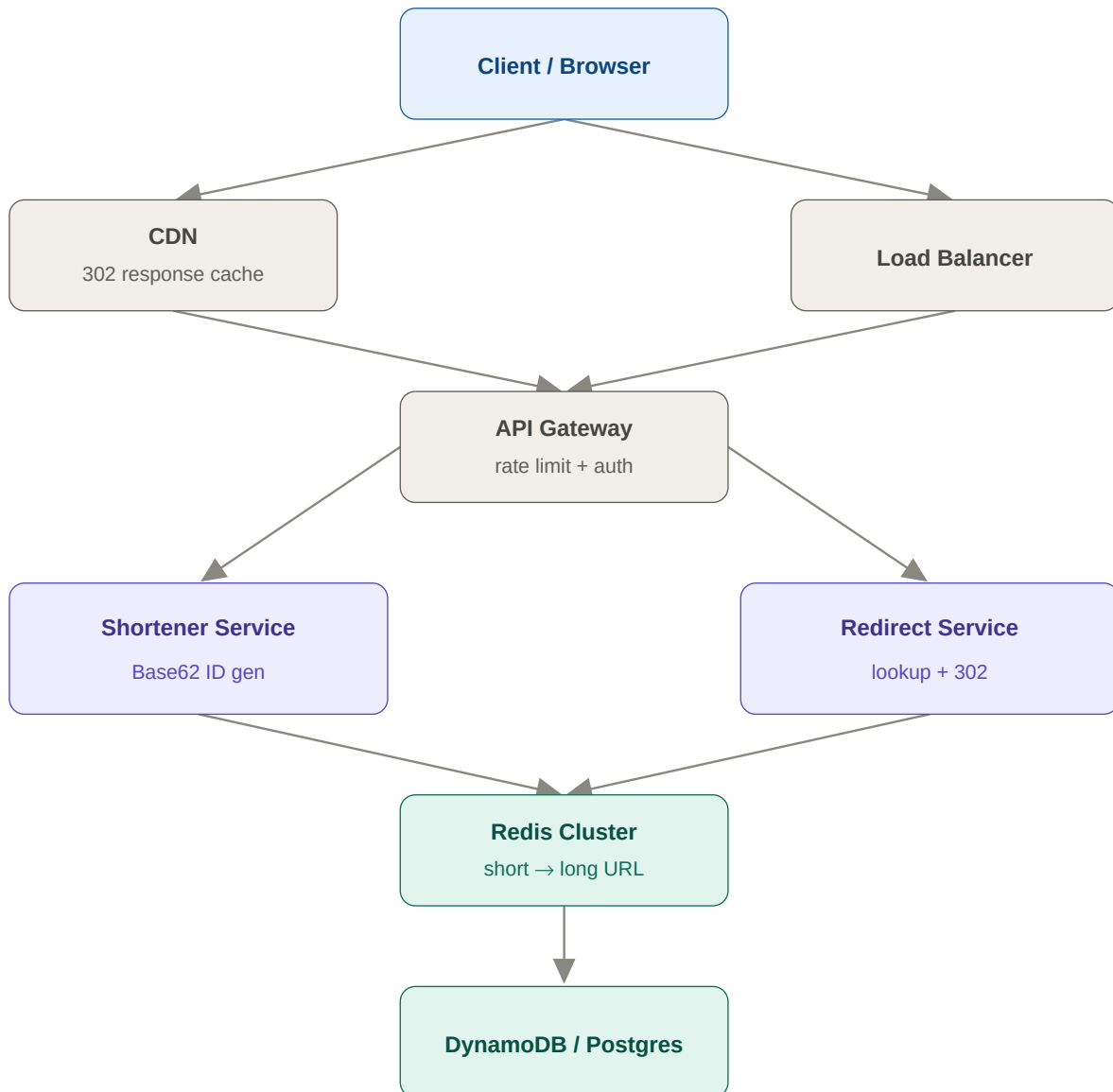
Think out loud: "Before I draw anything — let me clarify. Public service? Analytics? Custom aliases? Expiry? I'll assume yes to all."

- Functional: shorten URL, redirect, custom alias, expiry, click count
- Non-functional: <10ms redirect latency, 99.99% availability, no collision ever
- Scale: 100M new URLs/day, 10B redirects/day ≈ 116k RPS reads. Each URL ~500B → 18 TB/year

Scale	Users/RPS	Architecture	Storage
Low	1M URLs/day 1k RPS	Single app + Postgres + in-process cache	~200 GB/yr
Mid	10M/day 10k RPS	App cluster + Redis + Postgres + CDN	~2 TB/yr
High	100M/day 116k RPS	Stateless fleet + Redis cluster + DynamoDB + Global CDN	~18 TB/yr

2. Architecture Diagram

Think out loud: "Write path: Client → API GW → Shortener Service → ID Generator → DB. Read path hits CDN first, then Redis, then DB."



Architecture: solid = sync call. CDN caches 302 responses at edge, Redis caches lookup, DB is source of truth.

3. DB Schema

Think out loud: "Two tables: *urls* (the mapping) and *clicks* (analytics). *clicks* is append-only — great for Cassandra or a separate ClickHouse analytical store."

urls			clicks (analytics)		
PK	short_code	<i>VARCHAR(8)</i>	PK	click_id	<i>UUID</i>
	long_url	<i>TEXT</i>	FK	short_code	<i>VARCHAR(8)</i>
FK	user_id	<i>BIGINT</i>		clicked_at	<i>TIMESTAMP</i>
	created_at	<i>TIMESTAMP</i>		ip_hash	<i>VARCHAR(64)</i>
	expires_at	<i>TIMESTAMP</i>		country	<i>VARCHAR(3)</i>
	click_count	<i>BIGINT</i>		referrer	<i>TEXT</i>

Why this choice: short_code is the primary key — all reads are O(1) key lookups. We index long_url too to detect duplicate submissions and return the existing short code.

Why separate analytics: clicks table is deliberately separate. At 116k redirects/sec, writing click events inline would create a write bottleneck. We write asynchronously via Kafka → ClickHouse.

4. API & Trade-offs

POST	/api/v1/shorten	{long_url, alias?, ttl_days?} → {short_url}
GET	/[{short_code}]	302 redirect to long_url
GET	/api/v1/{short_code}/stats	Returns click count + geography
DELETE	/api/v1/{short_code}	Owner deletes their link

Decision	Choice	Gain	Trade-off
Code gen	Counter+Base62	Zero collisions	Sequential IDs (add random offset)
Redirect	302 not 301	We control expiry	Browser won't cache — extra hop
Cache	Redis cluster	<1ms lookup	Memory cost, eventual on update
Analytics	Async Kafka→CH	No write bottleneck	Slight delay in stats